Glue Connectivity to DocumentDB

- Purpose of the document
- Common libraries for Glue
  - Code repository
  - S3 location of the common libraries:
  - How to use the common library in Glue
- Sample code

Purpose of the document

Provides details around

- How to connect to the DocumentDB in AWS Glue jobs.
- Common libraries developed for AWS Glue jobs.

Common libraries for Glue

Code repository

scdhhs-mes-commonlibrary | AWS Developer Tools | us-east-1 (amazon.com)

S3 location of the common libraries:

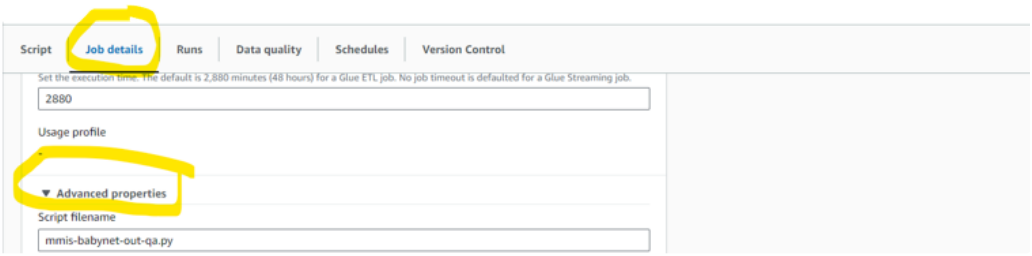
- s3://scdhhs-mmis-babynet-lower/dev/code/lib/commonPythonlib.zip

Common library was developed to connect to DocumentDB which will help to perform operations like read, write etc. Beauty of this is not to have any of the DocumentDB related information in the glue job and it is embedded in the common library. Centralized code helps us better in terms of code management, touching multiple jobs can be avoided for any future updates to the DocumentDB functions.

How to use the common library in Glue

In order to start using the common library and connect to DocumentDB, follow the below steps:

1. Import the common library in your glue job through advance properties using Job details tab.



2. Inside the glue job import library:

   from commonPythonlib import referenceData, readWriteOptions, commonFunctions

3. start using the libary and perform the operations

   a. **Write operation:**

      use below statement to write to DocumentDB.

      write_to_documentDB(glue_context,dataframe_input,secretParam,collectionName,type,log)

      glue_context: glue context

      dataframe_input: dataframe_input contains the data to be inserted into DocumentDB

      secretParam: secret manager name which contains credential information to connect to DocumentDB. Example: scdhhs-mes-qa/docdb

      collectionName: collection name to read from

      type: it can have have values of **raw** or anything else. if it is **raw**, database will be **rawDatabase** else it will be **canonicalDatabase**

      log: logger that will be used to log statements in the common library

   b. Read operation

      use below statement to read the data from DocumentDB to Json Array

      json_array = read_from_documentDB(glue_context,query,secretParam,collectionName,type,log)

   Inputs:

      glue_context: glue context

      query: query to perform to get the results.

         example: `{'$match': {'envelope.headers.metadata.batchId': 'BNET-d31c050a5b784286a6558cdd6e138747'}}`

secretParam: secret manager name which contains credential information to connect to DocumentDB. Example: scdhhs-mes-qa/docdb

collectionName: collection name to read from

type: it can have have values of **raw** or anything else. if it is **raw**, database will be **rawDatabase** else it will be **canonicalDatabase**

log: logger that will be used to log statements in the common library

Outputs:

it will return the json array with the results

Sample code

```
def write_to_documentDB(dataframe_input,resourceColName,collectionName):
    try:
        job = Job(glue_context)
        job.init(args['JOB_NAME'], args)
        #dataframe_input.show(10, truncate=False)
        #resourceColName = 'document'

        json_schema = spark.read.json(dataframe_input.rdd.map(lambda row:
row[f'{resourceColName}'])).schema
        records_list = dataframe_input.withColumn('json',
from_json(col(f'{resourceColName}'), json_schema)).drop(
            f'{resourceColName}')
        records_list = (records_list
                        #.withColumn("_id", col("json").getField("id"))
                         .withColumn("envelope", col("json").getField("envelope"))
                        .drop("json")
                        )
        log.info(f"Writing total babynet records to database :: {records_list.count()}")
        dyf = DynamicFrame.fromDF(records_list, glue_context, "dyf")


readWriteOptions.write_to_documentDB(glue_context,dyf,secretKeyName,collectionName,"canonica
l", log)
        log.info(f"Completed writing babynet transformed documents to DocumentDB")

        job.commit()
    except Exception as e:
        log.info(traceback.format_exc())
        log.error(f"Error occurred while writing to documentDB :: {e}")
        raise Exception(f"Error occurred while writing to db. Further details available in
cloudwatch ({glueJobName})")
```